Benchmarking Supervised Machine Learning Algorithms using PyCaret for Binary and Multiclassification of Network Anomaly Attacks

Manish Katel, Kiran Malla, Dr. Nick Rahimi

The University of Southern Mississippi

THE UNIVERSITY OF SOUTHERN MISSISSIPPI

Research Purpose

Goal

Design and evaluate the most efficient machine learning pipeline for detecting network anomaly attacks — achieving high accuracy with minimal computational cost, suitable for retraining on standard consumer hardware.

Binary Classification

Normal vs. Attack
(Benign vs. Anomaly)

Multi-Classification

Identify specific
attack type (9 classes)

Pipeline Efficiency

Memory-efficient,
no GPU required

Introduction: The Cybersecurity Landscape

~16 Billion

Cyberattacks attempted alone in 2025 (FBI)

- Cyberattacks evolve faster than traditional defenses
- Signature-based IDS fail against new/unknown threats
- ML-based Intrusion Detection Systems (IDS) are essential for real-time monitoring
- High-dimensional data, class imbalance & memory costs are key challenges

Supervised ML with proper preprocessing is necessary for accurate, efficient anomaly detection.

Signature vs. Anomaly-Based Detection

Signature-Based Detection	Anomaly-Based Detection (ML)
Relies on predefined attack signatures	Learns normal behavior patterns
Very effective against known malware	Detects novel and unknown attacks
Low computational cost	Learns complex feature patterns
No adaptation to evolving threats	Handles evolving traffic distributions
Low computational cost	Scalable with proper model selection

Network Attack Types (CICIDS-2017)

DoS / DDoS

Overwhelm servers/networks to deny service

Port Scan

Probe open ports to map vulnerabilities

Brute Force

Repeated login attempts (FTP, SSH)

Web Attack

XSS, SQL Injection, Brute Force via HTTP

Bot

Automated malicious activity, command & control

Infiltration

Attacker gains access inside the network

Heartbleed

SSL/TLS memory leak vulnerability exploit

Benign

Normal legitimate network traffic

ML-Based Network Anomaly Detection

Why Machine Learning?

Static Analysis

Examines network packets/logs without executing. Fast but limited to known patterns.

Dynamic Analysis

Monitors live behavior and traffic. Detects anomalies in real-time.

Supervised ML Advantages for IDS:

✓ Extracts discriminative patterns from labeled network traffic data

✓ Can detect deviations from normal behavior across multiple features

✓ Scalable to high-dimensional datasets (79 features in CICIDS-2017)

✓ AutoML (PyCaret) enables simultaneous comparison of 16+ algorithms

Dataset: CICIDS-2017

22.3M	79	15→9	80/20
Total Records	Features	Attack Classes (after grouping)	Benign / Attack Split

Key Dataset Features

- Flow Duration
- Total Fwd/Bwd Packets
- Packet Length Stats
- Flow IAT (inter-arrival times)
- Flag counts (SYN, ACK, FIN...)

⚠ Class Imbalance Challenge

80.30% Benign ▌ 19.70% Attack

Extreme imbalance between benign and attack traffic makes naive classifiers biased toward majority class.

Three-Layer Research Challenges

1. Dataset Challenges	2. ML Requirements	3. Research Constraints
▸ 80/20 class imbalance	▸ Sufficient & varied samples per class	▸ No GPU — standard consumer CPU only
▸ 22M+ records — memory pressure	▸ Balanced class distributions for training	▸ No Model Based Outlier Detection (slow)
▸ Inconsistent column names across 8 CSV files	▸ Feature quality & dimensionality	▸ No Model Based Feature Selection (too costly)
▸ Infinite/missing values	▸ Prevent data leakage & overfitting	▸ No hyperparameter tuning overhead

Like writing a poem without using certain words — every constraint made the solution more creative.

Data Preparation Pipeline

Data Integration
8 CSVs merged
Columns standardized

Data Cleaning
308K dupes removed
2,775 inf → NaN → median

Data Preprocessing
Label encoding
Z-score normalization

Feature Reduction
IPCA: 79→35 dims
99.3% variance kept

Data Sampling
Binary: Downsample
Multi: SMOTE

Memory-Efficient Technique Substitutions

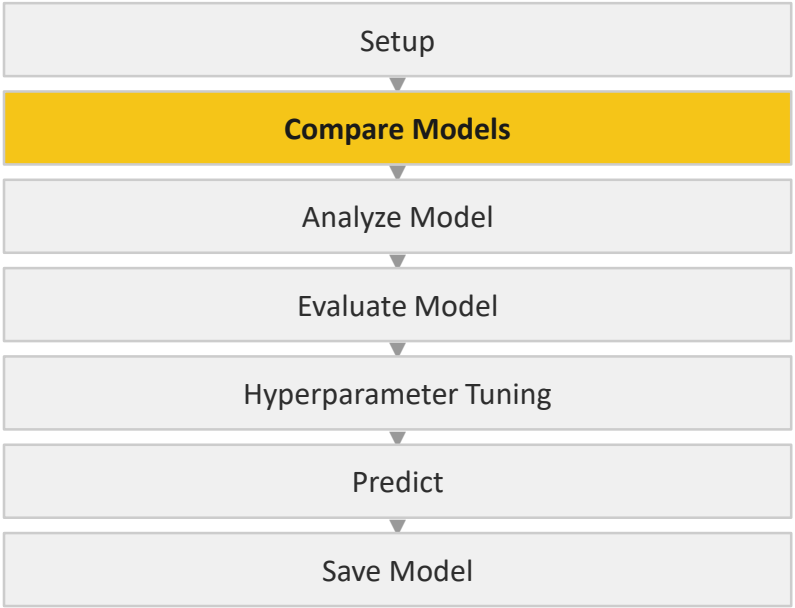
Standard Technique	Our Efficient Alternative	Reason
Standard PCA	Incremental PCA (IPCA)	Handles large data in chunks — memory efficient
Isolation Forest Outlier	Interquartile Range (IQR)	Similar accuracy, far less computation time
Model-Based Feature Selection	Correlation-Based Feature Selection	No training overhead required
Hyperparameter Tuning	PyCaret Model Comparison	Compare 16 models simultaneously, minimal code

PyCaret AutoML Framework

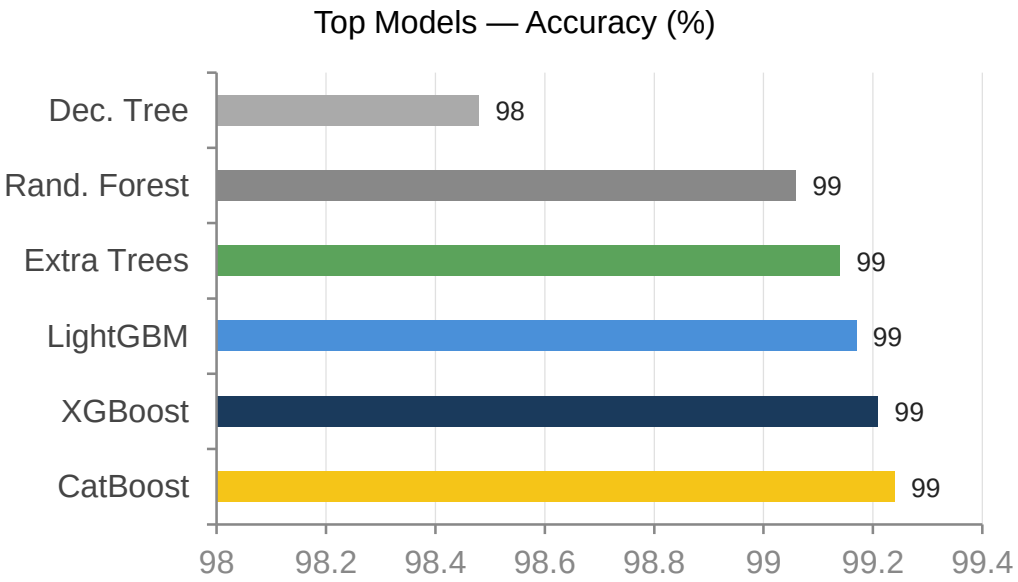
What is PyCaret?

- ✓ Low-code AutoML library in Python
- ✓ Integrates scikit-learn, XGBoost, LightGBM, CatBoost
- ✓ Compares up to 16 supervised algorithms at once
- ✓ Evaluates: Accuracy, Recall, Precision, F1, AUC, Training Time
- ✓ 10-fold stratified cross-validation built-in
- ✓ Enabled: preprocess=False (our custom pipeline used)

PyCaret Pipeline



Results: Binary Classification

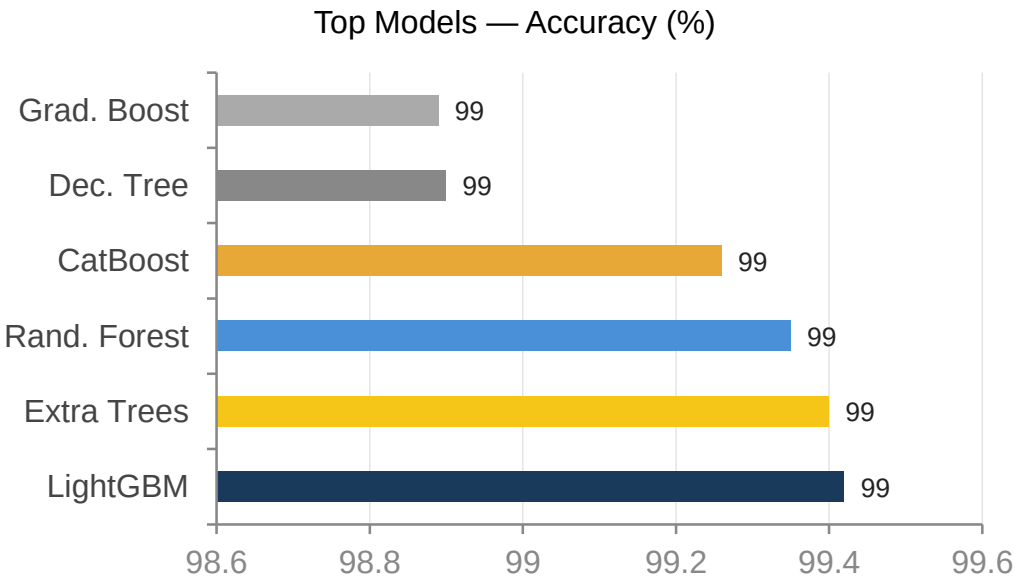


Top Models: Accuracy ≥ 99% + Training Time

Model	Accuracy	Time (s)
XGBoost ★	99.21%	0.158
LightGBM	99.17%	0.325
Extra Trees	99.14%	0.178
Random Forest	99.06%	0.682
CatBoost	99.24%	7.830

🏆 Best Overall: XGBoost
99.21% accuracy in just 0.158s — ideal for resource-constrained environments.

Results: Multi-Classification



Top Models: Accuracy ≥ 99% + Training Time

Model	Accuracy	Time (s)
Extra Trees ★	99.40%	0.670
LightGBM	99.42%	2.248
Random Forest	99.35%	2.713
CatBoost	99.26%	31.26

🏆 Best Balance: Extra Trees
99.40% accuracy in 0.670s.
Fastest among >99% accuracy models — ideal for rapid retraining.

Confusion Matrix Analysis — Attack-Specific Performance

Which model best detects each specific attack type?

Attack Type	Extra Trees	LightGBM	Random Forest	CatBoost	Best Model
BENIGN	98.29%	**98.38%** ★	98.08%	97.00%	LightGBM
Bot	99.88%	99.75%	99.79%	**99.96%** ★	CatBoost
Brute Force	99.75%	99.63%	**99.79%** ★	99.63%	Random Forest
DDoS	99.96%	99.96%	99.88%	**99.96%** ★	Tie (ET/LGBM/CB)
DoS	99.54%	99.54%	99.50%	**99.58%** ★	CatBoost
Port Scan	99.67%	99.71%	**99.75%** ★	99.75%	Random Forest
Web Attack	99.59%	99.58%	**99.79%** ★	99.63%	Random Forest

Insight: Ensemble of specialized classifiers per attack type may outperform any single model alone.

Comparison with Prior Work

Xu & Liu (2025) — CICIDS-2017 binary classification using MLP, CNN, LOF, OCSVM

Attack Type	Prior Best Model	Prior Accuracy	Our Best Model	Our Accuracy	Improvement
Web Attack	MLP	0.2500	Extra Trees	**0.9983**	**+74.83%** 🚀
Bot	LOF	0.4680	CatBoost	**0.9996**	**+53.16%** 🚀
DDoS	CNN	0.9996	ET/LGBM/CB	**0.9996**	0.00% (tie)
DoS	CNN	0.9978	LightGBM	**0.9954**	-0.24%
BENIGN	MLP	0.9974	LightGBM	**0.9838**	-1.36%
Port Scan	MLP	0.9995	Rand. Forest	**0.9975**	-0.20%
Brute Force	FTP-Patator	0.9987	Rand. Forest	**0.9979**	-0.08%

Our approach shows dramatically superior consistency: all categories >95% accuracy vs. prior work's 0.00–100% extreme variance. Seth et al. (2021) and Jin et al. (2020) also confirm that LightGBM, XGBoost & Extra Trees deliver fast, high-accuracy detection.

Overfitting & Pipeline Reliability Checks

Learning Curve Analysis	10-Fold Cross-Validation	No Data Leakage
Binary (CatBoost): Train ~99.9%, CV improves 98.4%→99.2% Multi (LightGBM): Train 100%, CV improves 98.8%→99.4% Narrow converging gap → no overfitting	Stratified folds ensure balanced class distribution Fair evaluation across all training/test splits Applied consistently to all 16 models	SMOTE applied only on training set after split Test set remains completely unseen Preprocessing done before split to preserve purity

- ✓ Hardware: AMD Ryzen 7 5825U, 16GB RAM — No GPU required
- ✓ All training done in standard CPU environment confirming real-world feasibility

Discussion & Future Directions

Key Discussion Points

- Tree-based ensemble models (XGBoost, LightGBM, CatBoost, Extra Trees) dominate both tasks
- Binary classification is easier — most models >98%
- Multi-class harder — similar patterns across attack types
- Clear accuracy-vs-time tradeoff: CatBoost best accuracy but slowest; Extra Trees best balance
- Downsampling + SMOTE addressed class imbalance effectively
- Pipeline successfully validated on malware detection dataset (80% on first run!)

Future Directions

- Hybrid supervised + unsupervised for zero-day attacks (Heartbleed, Infiltration)
- Real-time inference latency evaluation on streaming network data
- SHAP/LIME explainability to identify most important features
- Replace IPCA with explainability-preserving feature selection
- Test pipeline on more modern intrusion detection datasets

Limitations

Not Tested in Real-Time

All evaluations were performed on a static offline dataset. Deployment in production streaming environments has not been validated. Inference latency under real traffic loads is unknown.

IPCA Blocks Interpretability

Incremental PCA transforms original features into new components that don't map to real network features. SHAP / LIME feature importance analysis is therefore not applicable to this pipeline.

Rare Attacks Excluded

Infiltration (36 samples) and Heartbleed (11 samples) were excluded from SMOTE training due to insufficient data for reliable interpolation. These may appear in real-world traffic.

Hardware Dependency

Training Time (TT) is measured on AMD Ryzen 7 5825U + 16GB RAM. Results will vary across different hardware. The comparison is fair within experiment but may not generalize.

Conclusion

This study establishes a reproducible, memory-efficient ML pipeline for network intrusion detection using CICIDS-2017, demonstrating that AutoML frameworks like PyCaret enable comprehensive, fair model comparison without GPU infrastructure.

Binary Classification	**XGBoost**	99.21% accuracy · 0.158s
Multi-Classification	**Extra Trees**	99.40% accuracy · 0.670s
Best Benign Detection	**LightGBM**	98.38% per-class accuracy
Best Bot Detection	**CatBoost**	99.96% per-class accuracy
Best Brute Force/Port Scan/Web Attack	**Random Forest**	Up to 99.79% per-class

Tree-based ensemble models are the optimal choice for network anomaly detection on balanced datasets without requiring high-end computing resources.

Thank You

Questions & Discussion

Manish Katel · Kiran Malla · Dr. Nick Rahimi
The University of Southern Mississippi

Scan to read our paper

THE UNIVERSITY OF SOUTHERN MISSISSIPPI